$$\boldsymbol{n = 10}$$

Мінімізувати функцію $f\,(x) = 3\left(x_1 - 10\right)^2 - x_1 x_2 + 4x_2^2$

Початкова точка $x^{(0)} = \begin{pmatrix} -16 \\ -16 \end{pmatrix}$

Початкова система напрямків:

$$S^{(1)} = (1{,}0)^T,\ S^{(2)} = (0,\ 1)^T$$

Для алгоритма Свена $\lambda_0 = 0,\ \Delta\lambda\ =\ 0{,}1\ (\dfrac{\left\|x\left(k\right)\right\|}{\left\|s\left(k\right)\right\|})$

Система напрямків, за якою буде здійснюватися пошук:

$$S^{(2)},\ S^{(1)},\ S^{(2)},\ (x^3 - x^1)$$

Точність пошуку $\varepsilon = 0.01$

Ітерація 1.

$$x^1 = x^0 + \lambda_0\,(0{,}1)^T$$

$$\Delta\lambda = 0{,}1\left(\frac{\|\left(-16, -16\right)^T\|}{\|\,(0{,}1)^T\|}\right) = 2{,}2627$$

Інтервал невизначеності - [6,7882, 24,8902]

Метод золотого перерізу:

Інтервал - [13,997, 14,005]

$$\lambda_0 = 14{,}001$$

$$x^1 = \begin{pmatrix} -16 \\ -16 \end{pmatrix} + 14{,}001\,(0{,}1)^T = \begin{pmatrix} -16 \\ -1{,}999 \end{pmatrix}$$

Ітерація 2.

$$x^2 = x^1 + \lambda_1\,(1{,}0)^T$$

$$\Delta\lambda = 0{,}1\left(\frac{\|\left(-16, -1{,}9992\right)^T\|}{\|\,(1{,}0)^T\|}\right) = 1{,}6124$$

Інтервал невизначеності - [11,287, 34,086]

Метод золотого перерізу:

Інтервал - [25,6646, 25,6714]

$$\lambda_1 = 25{,}668$$

$$x^2 = \begin{pmatrix} -16 \\ -1{,}999 \end{pmatrix} + 25{,}668\,(1{,}0)^T = \begin{pmatrix} 9{,}668 \\ -1{,}999 \end{pmatrix}$$

Ітерація 3.

$$x^3 = x^2 + \lambda_2 (0,1)^T$$

$$\Delta\lambda = 0,1 \left(\frac{\| \left(9,668 - 1,9992\right)^T \|}{\| (1,0)^T \|} \right) = 0,9873$$

Інтервал невизначеності - [0,9873, 4,9363]

метод ДСК-Пауела

$$x_1 = 2,9617,\ x_2 = 3,2075,\ x_3 = 4,9363,\ \ x^* = 3,2075$$
$$\lambda_2 = 3,2075$$
$$x^3 = \begin{pmatrix} 9,668 \\ -1,999 \end{pmatrix} + 3,2075\,(0,1)^T = \begin{pmatrix} 9,668 \\ 1,2085 \end{pmatrix}$$

Ітерація 4.

$$x^4 = x^3 + \lambda_3 \left(x^3 - x^1\right)^T$$

$$\left(x^3 - x^1\right) = \begin{pmatrix} 9,668 \\ 1,2085 \end{pmatrix} - \begin{pmatrix} -16 \\ -1,999 \end{pmatrix} = \begin{pmatrix} 25,668 \\ 3,2075 \end{pmatrix}$$

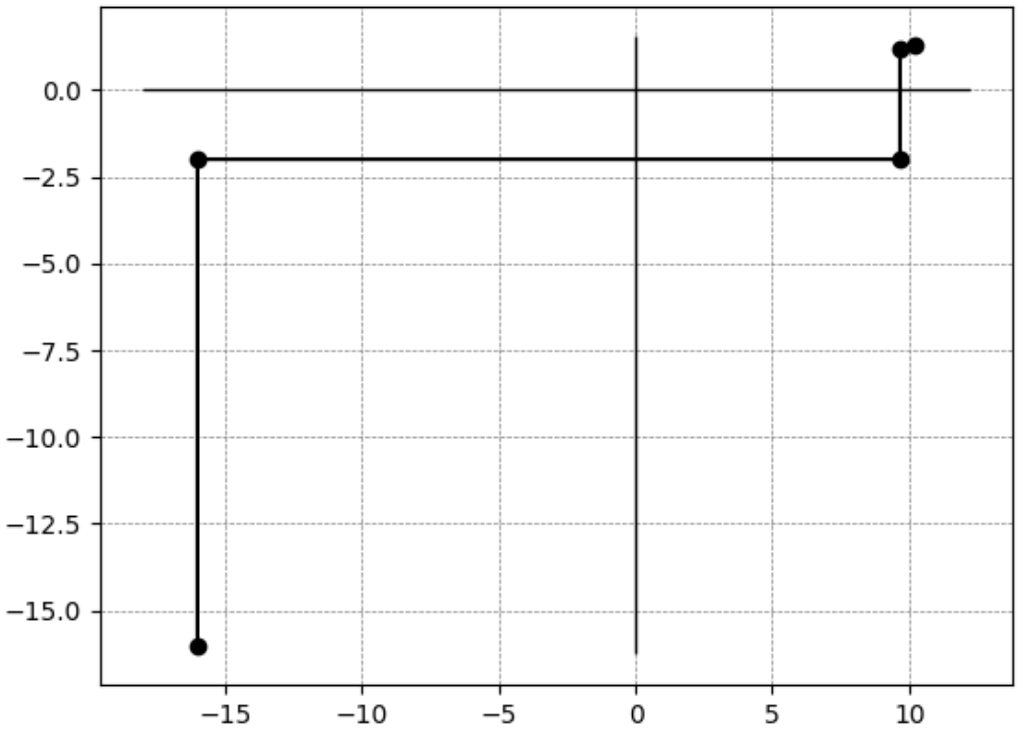
$$\Delta\lambda = 0,1 \left(\frac{\| \left(9,668,\ 1,2085\right)^T \|}{\| \left(25,668,\ 3,2075\right)^T \|} \right) = 0,0377$$

Інтервал невизначеності - [0, 0,0753]

метод ДСК-Пауела

$$x_1 = 0,\ x_2 = 0,0212,\ x_3 = 0,0377,\ \ x^* = 0,0212$$
$$\lambda_3 = 0,0212$$
$$x^4 = \begin{pmatrix} 9,668 \\ 1,2085 \end{pmatrix} + 0,0212 \begin{pmatrix} 25,668 \\ 3,2075 \end{pmatrix} = \begin{pmatrix} 10,2128 \\ 1,2766 \end{pmatrix}$$

Графік траєкторії пошуку:



Програма (якщо використовується):

```
import numpy as np
import matplotlib.pyplot as plt

n = 10
epsilon = 0.01

def f_2d(x):
    return 3 * (x[0] - 10)**2 - x[0] * x[1] + 4 * (x[1]**2)

x0 = np.array([-1.2 * n - 4, -1.2 * n - 4])
s1 = np.array([1, 0])
s2 = np.array([0, 1])


def get_f_1d(x_start, direction):
    """Reduces 2D function to 1D along a given direction from x_start."""
    def f_1d(lam):
        point_2d = x_start + lam * direction
        return f_2d(point_2d)
    return f_1d

lam0 = 0
def sven_algorithm(f_1d, x_vec, s_vec):
    d = 0.1 * (np.linalg.norm(x_vec) / np.linalg.norm(s_vec))

    y0 = f_1d(lam0)

    x_left = lam0 - d
    x_right = lam0 + d
    y_left = f_1d(x_left)
    y_right = f_1d(x_right)

    if y_left >= y0 and y_right >= y0:
        return [x_left, x_right]
```

```
    elif y_left < y0:
        sign = -1
        points = [lam0, x_left]
        funcs = [y0, y_left]
    else:
        sign = 1
        points = [lam0, x_right]
        funcs = [y0, y_right]

    count = 1
    while True:
        step = d * 2 ** count
        x_new = points[-1] + sign * step
        y_new = f_1d(x_new)

        if y_new < funcs[-1]:
            points.append(x_new)
            funcs.append(y_new)
            count += 1
        else:
            x_mid = (points[-1] + x_new) / 2
            points.append(x_mid)
            funcs.append(f_1d(x_mid))


            break

    a, b = sorted([points[-3], points[-1]])
    print(points)
    print(funcs)
    return [a, b]

def golden_ratio(f_1d, interval, eps):
    a, b = interval[0], interval[1]

    x1 = a + 0.382 * (b - a)
    x2 = a + 0.618 * (b - a)
    y1 = f_1d(x1)
    y2 = f_1d(x2)

    while b - a > eps:
        if y1 < y2:
            b = x2
            x2 = x1
            y2 = y1
            x1 = a + 0.382 * (b - a)
            y1 = f_1d(x1)
        else:
            a = x1
            x1 = x2
            y1 = y2
            x2 = a + 0.618 * (b - a)
            y2 = f_1d(x2)

    print(a, b)
    return (a + b) / 2

def dsk_powell(f_1d, interval, eps):
    x_mid = (interval[0] + interval[1]) / 2
    interval_p = [interval[0], x_mid, interval[1]]
    funcs = [f_1d(p) for p in interval_p]

    while True:
        a1 = (funcs[1] - funcs[0]) / (interval_p[1] - interval_p[0])
        a2 = (1 / (interval_p[2] - interval_p[1])) * (
            (funcs[2] - funcs[0]) / (interval_p[2] - interval_p[0])
            - (funcs[1] - funcs[0]) / (interval_p[1] - interval_p[0])
        )

        if a2 == 0:
            return interval_p[1]

        x_star = (interval_p[0] + interval_p[1]) / 2 - a1 / (2 * a2)
        y_star = f_1d(x_star)

        if abs(interval_p[1] - x_star) <= eps and abs(funcs[1] - y_star) <= eps:
            return x_star

        inserted = False
        for i in range(len(interval_p) - 1):
```

```
            if interval_p[i] <= x_star <= interval_p[i + 1]:
                interval_p.insert(i + 1, x_star)
                funcs.insert(i + 1, y_star)
                inserted = True
                print(interval_p, funcs)
                break

        if not inserted:
            if x_star < interval_p[0]:
                interval_p.insert(0, x_star)
                funcs.insert(0, y_star)
            else:
                interval_p.append(x_star)
                funcs.append(y_star)

        y_min = min(funcs)
        min_index = funcs.index(y_min)

        if min_index == 0:
            interval_p.pop(-1)
            funcs.pop(-1)
        elif min_index == len(funcs) - 1:
            interval_p.pop(0)
            funcs.pop(0)
        else:
            left_dist = abs(interval_p[min_index] - interval_p[0])
            right_dist = abs(interval_p[-1] - interval_p[min_index])
            if left_dist > right_dist:
                interval_p.pop(0)
                funcs.pop(0)
            else:
                interval_p.pop(-1)
                funcs.pop(-1)

history = [x0]

f_1d = get_f_1d(history[-1], s2)
interval = sven_algorithm(f_1d, history[-1], s2)
lam_star = golden_ratio(f_1d, interval, epsilon)
x1 = history[-1] + lam_star * s2
history.append(x1)
print(f"Iter 1: x1 = {x1}, lam = {lam_star:.4f}")

f_1d = get_f_1d(history[-1], s1)
interval = sven_algorithm(f_1d, history[-1], s1)
lam_star = golden_ratio(f_1d, interval, epsilon)
x2 = history[-1] + lam_star * s1
history.append(x2)
print(f"Iter 2: x2 = {x2}, lam = {lam_star:.4f}")

f_1d = get_f_1d(history[-1], s2)
interval = sven_algorithm(f_1d, history[-1], s2)
lam_star = dsk_powell(f_1d, interval, epsilon)
x3 = history[-1] + lam_star * s2
history.append(x3)
print(f"Iter 3: x3 = {x3}, lam = {lam_star:.4f}")

s_accel = history[3] - history[1]
f_1d = get_f_1d(history[-1], s_accel)
interval = sven_algorithm(f_1d, history[-1], s_accel)
lam_star = dsk_powell(f_1d, interval, epsilon)
x4 = history[-1] + lam_star * s_accel
history.append(x4)
print(f"Iter 4: x4 = {x4}, lam = {lam_star:.4f}")

print("\nFinal Result:", x4)
print(f"f(x4) = {f_2d(x4):.6f}")

x_coords = [point[0] for point in history]
y_coords = [point[1] for point in history]

plt.plot(x_coords, y_coords, color='black', marker='o')
plt.vlines(0, min(y_coords) - 0.2 * max(y_coords), max(y_coords) + 0.2 *
max(y_coords), color='black', linewidth=1)
plt.hlines(0, min(x_coords) - 0.2 * max(x_coords), max(x_coords) + 0.2 *
max(x_coords), color='black', linewidth=1)
plt.grid(True, linestyle='--', color='gray', linewidth=0.5)
plt.show()
```